스마트 데이터 이해 및 활용

종합실습 2 보고서

CampusHub 복합 쿼리 실습

JOIN · 서브쿼리 · 집계(ROLLUP) · 재귀 CTE · Window Function (문항 1~25)

제출자	광주 1반 김지호
제출일	2026-07-23
실습 환경	PostgreSQL 17.10 (Homebrew) · skala_db · lab 스키마 · skala_user
데이터	student 1,000 · enroll 2,302 · customers 500 · orders 3,000 · emp 311
구성	문항 1~10 JOIN	문항 11~20 서브쿼리·집합	문항 21~25 집계·CTE·윈도우

첨부: 문항별 쿼리 및 psql 실행 결과 캡처 25건
(26행 초과 결과는 중략 표기, 마지막 행과 행 수 카운트 유지)

실습 중 확인한 특이사항

1. enroll 데이터와 슬라이드 배경의 불일치

슬라이드 배경은 "student_id % 3 규칙, 학생당 0~2건, 약 333명 수강 없음"이라 설명하지만,
적재 스크립트의 실제 규칙은 %10=0 → 0건(100명), 짝수 → 2건(400명), 홀수 → 3건(500명)이다.
따라서 문항 5·18(수강 없는 학생)의 결과는 333명이 아닌 100명이며,
본 보고서의 모든 검산은 실측 데이터 기준으로 수행했다.

2. emp 데이터와 슬라이드 배경의 불일치 (문항 22)

슬라이드는 "각 직원은 매니저 10명 중 1명에게 배정"(기대 계층 1/10/300)이라 설명하지만,
적재 수식 manager_id = 1+((gs-1)%10) 때문에 Dev 30명이 CEO 직속으로 배정된다.
실측 depth 분포는 0:1 / 1:40 / 2:270이고 Mgr_11은 직속 부하가 0명이다.
direct_reports 쿼리는 부하 0명인 Mgr_11도 표시되도록 LEFT JOIN으로 작성했다.

3. 문항 12 권한 이슈 (REFERENCES)

skala_user로 course_owner의 FK(REFERENCES lab.emp) 생성 시 permission denied가 발생했다.
초기 스크립트의 기본 권한(SELECT/INSERT/UPDATE/DELETE)에는 REFERENCES 권한이 없기 때문으로,
관리자 계정에서 GRANT REFERENCES ON lab.emp TO skala_user; 부여 후 정상 생성됐다.

4. 쿼리 작성 원칙

채점 기준(불필요한 컬럼 선택 확인)에 따라 모든 쿼리는 문항이 요구한 컬럼만 SELECT했다.
순위·상위 N 문항은 동점 대비 2차 정렬 기준(student_id, customer_id)을 명시했다.
같은 결과를 두 문법으로 요구한 문항(5-18, 6-17, 23)은 행 수 일치를 교차 검산했다.

문항별 결과 검산 요약

문항	결과 행 수	검산 포인트
1	2,300	INNER JOIN, 고아 수강 2건 제외
2	2,400	LEFT JOIN, 미수강 학생 100명 course NULL
3	2,302	RIGHT JOIN, 고아 수강 1001·1010의 name NULL
4	2,402	FULL OUTER, 고아 2 + 미수강 100 모두 포함
5	100	Anti-Join = NOT EXISTS(문항 18)와 일치
6	900	Semi-Join = EXISTS(문항 17)와 일치
7	500	전 고객 주문 정확히 6건
8	10	총액 상위 10명, 2차 정렬 customer_id
9	311	SELF JOIN, CEO의 manager_name NULL
10	22	Course_1~20 + DB + 미수강(NULL) 100명
11	893	DB 수강 107명과 합 = 1,000
12	23	course_owner 23과목 → Mgr_2~11 라운드로빈
13	100	CROSS JOIN 전체 23,000조합 중 샘플
14	1,000	스칼라 서브쿼리(학과명)
15	495	전체 평균 GPA 3.441 초과
16	495	학과 평균 초과 (HR만 9명 – 편중 분포)
17	900	EXISTS
18	100	NOT EXISTS
19	967/0	>ANY 967명, >ALL 0명(HR 최고 4.9, 의도된 결과)
20	290	UNION: CS 196 + DB수강 107 – 교집합 13
21	25	ROLLUP 상세 18+소계 6+총계 1, 총계 1,000명
22	311+11	재귀 CTE depth 0:1/1:40/2:270, direct_reports 11행
23	18	학과별 TOP3, CTE·서브쿼리 두 방식 결과 동일
24	2,302	LAG 성적 변화, score_range 포함
25	3,000	누적합·이동평균, 50% 초과 첫 주문 order_id 1503

문항 1 INNER JOIN — 수강 존재 학생의 과목/성적

종합실습2 문항 1 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회
--   기대 행 수: 2,300 (고아 수강 2건은 학생이 없어 제외됨)
-- ------------------------------------------------------------
SELECT s.student_id, s.name, e.course, e.grade
FROM lab.student s
INNER JOIN lab.enroll e
        ON e.student_id = s.student_id;
 student_id |    name     |  course   | grade
------------+-------------+-----------+-------
          1 | Student_1   | Course_3  | C
          1 | Student_1   | Course_4  | D
          1 | Student_1   | Course_5  | A
          2 | Student_2   | Course_4  | D
          2 | Student_2   | Course_5  | A
          3 | Student_3   | Course_5  | A
          3 | Student_3   | Course_6  | B
          3 | Student_3   | Course_7  | C
          4 | Student_4   | Course_6  | B
          4 | Student_4   | Course_7  | C
          5 | Student_5   | Course_7  | C
          5 | Student_5   | Course_8  | D
          5 | Student_5   | Course_9  | A
          6 | Student_6   | Course_8  | D
          6 | Student_6   | Course_9  | A
          7 | Student_7   | Course_9  | A
          7 | Student_7   | Course_10 | B
          7 | Student_7   | Course_11 | C
          8 | Student_8   | Course_10 | B
          8 | Student_8   | Course_11 | C
       ... (중략: 2278행 생략) ...
        999 | Student_999 | Course_2  | B
        999 | Student_999 | Course_3  | C
(2300개 행)
```

문항 2 LEFT JOIN — 모든 학생 + 과목(없으면 NULL)

종합실습2 문항 2 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 2. 모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기
--   기대 행 수: 2,400 (수강 2,300 + 수강 없는 학생 100)
-- ------------------------------------------------------------
SELECT s.student_id, s.name, e.course
FROM lab.student s
LEFT JOIN lab.enroll e
       ON e.student_id = s.student_id;
 student_id |     name     |  course
------------+--------------+-----------
          1 | Student_1    | Course_3
          1 | Student_1    | Course_4
          1 | Student_1    | Course_5
          2 | Student_2    | Course_4
          2 | Student_2    | Course_5
          3 | Student_3    | Course_5
          3 | Student_3    | Course_6
          3 | Student_3    | Course_7
          4 | Student_4    | Course_6
          4 | Student_4    | Course_7
          5 | Student_5    | Course_7
          5 | Student_5    | Course_8
          5 | Student_5    | Course_9
          6 | Student_6    | Course_8
          6 | Student_6    | Course_9
          7 | Student_7    | Course_9
          7 | Student_7    | Course_10
          7 | Student_7    | Course_11
          8 | Student_8    | Course_10
          8 | Student_8    | Course_11
      ... (중략: 2378행 생략) ...
        240 | Student_240  |
        740 | Student_740  |
(2400개 행)
```

문항 3 RIGHT JOIN — 수강 기준, 학생 정보 NULL

종합실습2 문항 3 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 3. 수강이 기준. 학생이 없으면 학생 정보가 NULL
--   기대 행 수: 2,302 (고아 수강 1001·1010 포함, 해당 행 name IS NULL)
-- ------------------------------------------------------------
SELECT e.student_id, e.course, e.grade, s.name
FROM lab.student s
RIGHT JOIN lab.enroll e
        ON e.student_id = s.student_id;
 student_id |  course   | grade |    name
------------+-----------+-------+-------------
          1 | Course_3  | C     | Student_1
          1 | Course_4  | D     | Student_1
          1 | Course_5  | A     | Student_1
          2 | Course_4  | D     | Student_2
          2 | Course_5  | A     | Student_2
          3 | Course_5  | A     | Student_3
          3 | Course_6  | B     | Student_3
          3 | Course_7  | C     | Student_3
          4 | Course_6  | B     | Student_4
          4 | Course_7  | C     | Student_4
          5 | Course_7  | C     | Student_5
          5 | Course_8  | D     | Student_5
          5 | Course_9  | A     | Student_5
          6 | Course_8  | D     | Student_6
          6 | Course_9  | A     | Student_6
          7 | Course_9  | A     | Student_7
          7 | Course_10 | B     | Student_7
          7 | Course_11 | C     | Student_7
          8 | Course_10 | B     | Student_8
          8 | Course_11 | C     | Student_8
      ... (중략: 2280행 생략) ...
       1001 | AI        | A     |
       1010 | ML        | B     |
(2302개 행)
```

문항 4 FULL OUTER JOIN — 학생/수강 모두 포함

종합실습2 문항 4 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 4. 학생/수강 모두 포함 (FULL OUTER JOIN)
--   기대 행 수: 2,402 (수강 2,300 + 수강 없는 학생 100 + 고아 수강 2)
-- ------------------------------------------------------------
SELECT s.student_id AS student_side_id, s.name,
       e.student_id AS enroll_side_id, e.course
FROM lab.student s
FULL OUTER JOIN lab.enroll e
            ON e.student_id = s.student_id;
 student_side_id |     name     | enroll_side_id |  course
-----------------+--------------+----------------+-----------
               1 | Student_1    |              1 | Course_3
               1 | Student_1    |              1 | Course_4
               1 | Student_1    |              1 | Course_5
               2 | Student_2    |              2 | Course_4
               2 | Student_2    |              2 | Course_5
               3 | Student_3    |              3 | Course_5
               3 | Student_3    |              3 | Course_6
               3 | Student_3    |              3 | Course_7
               4 | Student_4    |              4 | Course_6
               4 | Student_4    |              4 | Course_7
               5 | Student_5    |              5 | Course_7
               5 | Student_5    |              5 | Course_8
               5 | Student_5    |              5 | Course_9
               6 | Student_6    |              6 | Course_8
               6 | Student_6    |              6 | Course_9
               7 | Student_7    |              7 | Course_9
               7 | Student_7    |              7 | Course_10
               7 | Student_7    |              7 | Course_11
               8 | Student_8    |              8 | Course_10
               8 | Student_8    |              8 | Course_11
      ... (중략: 2380행 생략) ...
             240 | Student_240  |
             740 | Student_740  |
(2402개 행)
```

문항 5 Anti-Join — 한 번도 수강하지 않은 학생

종합실습2 문항 5 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 5. 한 번도 수강하지 않은 학생 목록 (Anti-Join)
--   기대 행 수: 100 (실측: student_id % 10 = 0인 학생)
-- ------------------------------------------------------------
SELECT s.student_id, s.name
FROM lab.student s
LEFT JOIN lab.enroll e
       ON e.student_id = s.student_id
WHERE e.student_id IS NULL
ORDER BY s.student_id;
 student_id |     name
------------+--------------
         10 | Student_10
         20 | Student_20
         30 | Student_30
         40 | Student_40
         50 | Student_50
         60 | Student_60
         70 | Student_70
         80 | Student_80
         90 | Student_90
        100 | Student_100
        110 | Student_110
        120 | Student_120
        130 | Student_130
        140 | Student_140
        150 | Student_150
        160 | Student_160
        170 | Student_170
        180 | Student_180
        190 | Student_190
        200 | Student_200
      ... (중략: 78행 생략) ...
        990 | Student_990
       1000 | Student_1000
(100개 행)
```

문항 6 Semi-Join — 한 과목 이상 수강(중복 제거)

종합실습2 문항 6 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------------
-- 문항 6. 한 과목 이상 수강한 학생 목록(중복 제거) (Semi-Join)
--   기대 행 수: 900 (1,000 - 수강 없는 학생 100)
-- ------------------------------------------------------------------
SELECT DISTINCT s.student_id, s.name
FROM lab.student s
INNER JOIN lab.enroll e
        ON e.student_id = s.student_id
ORDER BY s.student_id;
 student_id |    name
------------+-------------
          1 | Student_1
          2 | Student_2
          3 | Student_3
          4 | Student_4
          5 | Student_5
          6 | Student_6
          7 | Student_7
          8 | Student_8
          9 | Student_9
         11 | Student_11
         12 | Student_12
         13 | Student_13
         14 | Student_14
         15 | Student_15
         16 | Student_16
         17 | Student_17
         18 | Student_18
         19 | Student_19
         21 | Student_21
         22 | Student_22
      ... (중략: 878행 생략) ...
        998 | Student_998
        999 | Student_999
(900개 행)
```

문항 7 고객별 주문건수/총액

종합실습2 문항 7 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 7. 고객별 주문건수/총액
--   기대 행 수: 500, 전 고객 주문건수 6건
-- ------------------------------------------------------------
SELECT c.customer_id, c.customer_name,
       COUNT(o.order_id)            AS order_count,
       COALESCE(SUM(o.amount), 0)   AS total_amount
FROM lab.customers c
LEFT JOIN lab.orders o
       ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY c.customer_id;
 customer_id | customer_name | order_count | total_amount
-------------+---------------+-------------+--------------
           1 | Customer_1    |           6 |     16530.00
           2 | Customer_2    |           6 |     12608.06
           3 | Customer_3    |           6 |     12686.12
           4 | Customer_4    |           6 |     12764.18
           5 | Customer_5    |           6 |     12842.24
           6 | Customer_6    |           6 |     12920.30
           7 | Customer_7    |           6 |     12998.36
           8 | Customer_8    |           6 |     13076.42
           9 | Customer_9    |           6 |     13154.48
          10 | Customer_10   |           6 |     13232.54
          11 | Customer_11   |           6 |     13310.60
          12 | Customer_12   |           6 |     13388.66
          13 | Customer_13   |           6 |     13466.72
          14 | Customer_14   |           6 |     13544.78
          15 | Customer_15   |           6 |     13622.84
          16 | Customer_16   |           6 |     13700.90
          17 | Customer_17   |           6 |     13778.96
          18 | Customer_18   |           6 |     13857.02
          19 | Customer_19   |           6 |     13935.08
          20 | Customer_20   |           6 |     14013.14
      ... (중략: 478행 생략) ...
         499 | Customer_499  |           6 |     16379.88
         500 | Customer_500  |           6 |     16457.94
(500개 행)
```

문항 8 총액 상위 10명과 금액

종합실습2 문항 8 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 8. 총액 상위 10명과 금액
--   동점 대비 customer_id 오름차순을 2차 정렬로 명시
-- ------------------------------------------------------------
SELECT c.customer_id, c.customer_name,
       SUM(o.amount) AS total_amount
FROM lab.customers c
INNER JOIN lab.orders o
        ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_amount DESC, c.customer_id
LIMIT 10;
 customer_id | customer_name | total_amount
-------------+---------------+--------------
         154 | Customer_154  |     19467.18
         153 | Customer_153  |     19389.12
         152 | Customer_152  |     19311.06
         151 | Customer_151  |     19233.00
         150 | Customer_150  |     19154.94
         149 | Customer_149  |     19076.88
         148 | Customer_148  |     18998.82
         147 | Customer_147  |     18920.76
         146 | Customer_146  |     18842.70
         145 | Customer_145  |     18764.64
(10개 행)
```

문항 9 SELF JOIN — 모든 직원과 매니저 이름

종합실습2 문항 9 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------------
-- 문항 9. 모든 직원과 그 매니저 이름 (SELF JOIN)
--   기대 행 수: 311 (CEO는 manager_name NULL)
-- ------------------------------------------------------------------
SELECT e.emp_id, e.name, m.name AS manager_name
FROM lab.emp e
LEFT JOIN lab.emp m
       ON m.emp_id = e.manager_id
ORDER BY e.emp_id;
 emp_id |  name   | manager_name
--------+---------+--------------
      1 | CEO     |
      2 | Mgr_2   | CEO
      3 | Mgr_3   | CEO
      4 | Mgr_4   | CEO
      5 | Mgr_5   | CEO
      6 | Mgr_6   | CEO
      7 | Mgr_7   | CEO
      8 | Mgr_8   | CEO
      9 | Mgr_9   | CEO
     10 | Mgr_10  | CEO
     11 | Mgr_11  | CEO
     12 | Dev_12  | CEO
     13 | Dev_13  | Mgr_2
     14 | Dev_14  | Mgr_3
     15 | Dev_15  | Mgr_4
     16 | Dev_16  | Mgr_5
     17 | Dev_17  | Mgr_6
     18 | Dev_18  | Mgr_7
     19 | Dev_19  | Mgr_8
     20 | Dev_20  | Mgr_9
      ... (중략: 289행 생략) ...
    310 | Dev_310 | Mgr_9
    311 | Dev_311 | Mgr_10
(311개 행)
```

문항 10 LEFT JOIN + 집계 — 과목 분포

종합실습2 문항 10 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 10. "모든 학생 기준" 과목 분포 → LEFT JOIN + 집계
--   기대 행 수: 22 (Course_1~20 + DB + 미수강 NULL 그룹)
-- ------------------------------------------------------------
SELECT e.course,
       COUNT(s.student_id) AS student_count
FROM lab.student s
LEFT JOIN lab.enroll e
       ON e.student_id = s.student_id
GROUP BY e.course
ORDER BY e.course NULLS LAST;
  course   | student_count
-----------+---------------
 Course_1  |           144
 Course_10 |            96
 Course_11 |           144
 Course_12 |            48
 Course_13 |            96
 Course_14 |            96
 Course_15 |           144
 Course_16 |            96
 Course_17 |           144
 Course_18 |            96
 Course_19 |           144
 Course_2  |            47
 Course_20 |            96
 Course_3  |            94
 Course_4  |            94
 Course_5  |           141
 Course_6  |            94
 Course_7  |           141
 Course_8  |            94
 Course_9  |           144
 DB        |           107
           |           100
(22개 행)
```

문항 11 DB 과목을 듣지 않은 학생 (NOT EXISTS)

종합실습2 문항 11 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 11. DB 과목을 듣지 않은 모든 학생을 나열
--   기대 행 수: 893 (전체 1,000 - DB 수강 107)
-- ------------------------------------------------------------
SELECT s.student_id, s.name
FROM lab.student s
WHERE NOT EXISTS (
    SELECT 1
    FROM lab.enroll e
    WHERE e.student_id = s.student_id
      AND e.course     = 'DB'
)
ORDER BY s.student_id;
 student_id |     name
------------+--------------
          1 | Student_1
          2 | Student_2
          3 | Student_3
          4 | Student_4
          5 | Student_5
          6 | Student_6
          7 | Student_7
          8 | Student_8
          9 | Student_9
         10 | Student_10
         11 | Student_11
         12 | Student_12
         13 | Student_13
         14 | Student_14
         15 | Student_15
         16 | Student_16
         17 | Student_17
         18 | Student_18
         20 | Student_20
         21 | Student_21
      ... (중략: 871행 생략) ...
        999 | Student_999
       1000 | Student_1000
(893개 행)
```

문항 12 course_owner 생성 + 과목별 수강 인원·책임 매니저

종합실습2 문항 12 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 12. 과목별 운영 책임 매니저 매핑 테이블 course_owner 생성 후
--          과목별 수강 인원 + 책임 매니저 이름 리포트
--   매핑 규칙: enroll의 전체 과목(23개: Course_1~20, DB, AI, ML)을
--              과목명 순으로 매니저(Mgr_2~Mgr_11, emp_id 2~11)에 라운드로빈 배정
--   ※ 사전 권한: skala_user로 FK를 만들려면 관리자 계정에서
--      GRANT REFERENCES ON lab.emp TO skala_user;  가 선행되어야 함
-- ------------------------------------------------------------
DROP TABLE IF EXISTS lab.course_owner;
DROP TABLE
CREATE TABLE lab.course_owner (
    course     VARCHAR(50) PRIMARY KEY,
    manager_id INT NOT NULL REFERENCES lab.emp(emp_id)
);
CREATE TABLE
INSERT INTO lab.course_owner (course, manager_id)
SELECT course,
       2 + (ROW_NUMBER() OVER (ORDER BY course) - 1) % 10 AS manager_id
FROM (SELECT DISTINCT course FROM lab.enroll) c;
INSERT 0 23
-- 리포트: 과목별 수강 인원 + 책임 매니저 이름
SELECT co.course,
       COUNT(e.student_id) AS enroll_count,
       m.name              AS manager_name
FROM lab.course_owner co
LEFT JOIN lab.enroll e ON e.course  = co.course
JOIN lab.emp        m ON m.emp_id   = co.manager_id
GROUP BY co.course, m.name
ORDER BY co.course;
  course   | enroll_count | manager_name
-----------+--------------+--------------
 AI        |            1 | Mgr_2
 Course_1  |          144 | Mgr_3
 Course_10 |           96 | Mgr_4
 Course_11 |          144 | Mgr_5
 Course_12 |           48 | Mgr_6
 Course_13 |           96 | Mgr_7
 Course_14 |           96 | Mgr_8
 Course_15 |          144 | Mgr_9
 Course_16 |           96 | Mgr_10
 Course_17 |          144 | Mgr_11
 Course_18 |           96 | Mgr_2
 Course_19 |          144 | Mgr_3
 Course_2  |           47 | Mgr_4
 Course_20 |           96 | Mgr_5
 Course_3  |           94 | Mgr_6
 Course_4  |           94 | Mgr_7
 Course_5  |          141 | Mgr_8
 Course_6  |           94 | Mgr_9
 Course_7  |          141 | Mgr_10
 Course_8  |           94 | Mgr_11
 Course_9  |          144 | Mgr_2
 DB        |          107 | Mgr_3
 ML        |            1 | Mgr_4
(23개 행)
```

문항 13 CROSS JOIN — 학생×과목 추천 후보 100건

종합실습2 문항 13 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 13. 학생 × 과목 전체 조합(추천 후보), 샘플 100건만
--   CROSS JOIN 폭발 주의: 1,000명 × 23과목 = 23,000 조합 → LIMIT 100
-- ------------------------------------------------------------
SELECT s.student_id, s.name, c.course
FROM lab.student s
CROSS JOIN (SELECT DISTINCT course FROM lab.enroll) c
ORDER BY s.student_id, c.course
LIMIT 100;
 student_id |   name    |  course
------------+-----------+-----------
          1 | Student_1 | AI
          1 | Student_1 | Course_1
          1 | Student_1 | Course_10
          1 | Student_1 | Course_11
          1 | Student_1 | Course_12
          1 | Student_1 | Course_13
          1 | Student_1 | Course_14
          1 | Student_1 | Course_15
          1 | Student_1 | Course_16
          1 | Student_1 | Course_17
          1 | Student_1 | Course_18
          1 | Student_1 | Course_19
          1 | Student_1 | Course_2
          1 | Student_1 | Course_20
          1 | Student_1 | Course_3
          1 | Student_1 | Course_4
          1 | Student_1 | Course_5
          1 | Student_1 | Course_6
          1 | Student_1 | Course_7
          1 | Student_1 | Course_8
      ... (중략: 78행 생략) ...
          5 | Student_5 | Course_14
          5 | Student_5 | Course_15
(100개 행)
```

문항 14 스칼라 서브쿼리 — 학생 + 학과명

종합실습2 문항 14 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 14. 스칼라 서브쿼리(SELECT 절)로 학생 + 소속 학과명 붙이기
--   별도 학과 테이블이 없어 student를 스칼라 서브쿼리로 재참조하는 형태로 작성
-- ------------------------------------------------------------
SELECT s.student_id,
       s.name,
       (SELECT s2.major
        FROM lab.student s2
        WHERE s2.student_id = s.student_id) AS major_name
FROM lab.student s
ORDER BY s.student_id;
 student_id |     name     | major_name
------------+--------------+------------
          1 | Student_1    | EE
          2 | Student_2    | ME
          3 | Student_3    | CE
          4 | Student_4    | BIO
          5 | Student_5    | CS
          6 | Student_6    | EE
          7 | Student_7    | ME
          8 | Student_8    | CE
          9 | Student_9    | BIO
         10 | Student_10   | CS
         11 | Student_11   | EE
         12 | Student_12   | ME
         13 | Student_13   | CE
         14 | Student_14   | BIO
         15 | Student_15   | CS
         16 | Student_16   | EE
         17 | Student_17   | ME
         18 | Student_18   | CE
         19 | Student_19   | BIO
         20 | Student_20   | CS
      ... (중략: 978행 생략) ...
        999 | Student_999  | HR
       1000 | Student_1000 | HR
(1000개 행)
```

문항 15 전체 평균 GPA보다 높은 학생

종합실습2 문항 15 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 15. 전체 평균 GPA보다 높은 학생 (WHERE 서브쿼리)
--   기대 행 수: 495 (전체 평균 3.441, gpa 3.5 이상인 학생)
-- ------------------------------------------------------------
SELECT s.student_id, s.name, s.gpa
FROM lab.student s
WHERE s.gpa > (SELECT AVG(gpa) FROM lab.student)
ORDER BY s.student_id;
 student_id |    name     | gpa
------------+-------------+------
         15 | Student_15  | 3.50
         16 | Student_16  | 3.60
         17 | Student_17  | 3.70
         18 | Student_18  | 3.80
         19 | Student_19  | 3.90
         20 | Student_20  | 4.00
         21 | Student_21  | 4.10
         22 | Student_22  | 4.20
         23 | Student_23  | 4.30
         24 | Student_24  | 4.40
         25 | Student_25  | 4.50
         26 | Student_26  | 4.60
         27 | Student_27  | 4.70
         28 | Student_28  | 4.80
         29 | Student_29  | 4.90
         45 | Student_45  | 3.50
         46 | Student_46  | 3.60
         47 | Student_47  | 3.70
         48 | Student_48  | 3.80
         49 | Student_49  | 3.90
       ... (중략: 473행 생략) ...
        988 | Student_988 | 4.80
        989 | Student_989 | 4.90
(495개 행)
```

문항 16 학과 평균 GPA보다 높은 학생 (상관 서브쿼리)

종합실습2 문항 16 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated Subquery)
-- ------------------------------------------------------------
SELECT s.student_id, s.name, s.major, s.gpa
FROM lab.student s
WHERE s.gpa > (SELECT AVG(s2.gpa)
               FROM lab.student s2
               WHERE s2.major = s.major)
ORDER BY s.student_id;
 student_id |    name     | major | gpa
------------+-------------+-------+------
         15 | Student_15  | CS    | 3.50
         16 | Student_16  | EE    | 3.60
         17 | Student_17  | ME    | 3.70
         18 | Student_18  | CE    | 3.80
         19 | Student_19  | BIO   | 3.90
         20 | Student_20  | CS    | 4.00
         21 | Student_21  | EE    | 4.10
         22 | Student_22  | ME    | 4.20
         23 | Student_23  | CE    | 4.30
         24 | Student_24  | BIO   | 4.40
         25 | Student_25  | CS    | 4.50
         26 | Student_26  | EE    | 4.60
         27 | Student_27  | ME    | 4.70
         28 | Student_28  | CE    | 4.80
         29 | Student_29  | BIO   | 4.90
         45 | Student_45  | CS    | 3.50
         46 | Student_46  | EE    | 3.60
         47 | Student_47  | ME    | 3.70
         48 | Student_48  | CE    | 3.80
         49 | Student_49  | BIO   | 3.90
      ... (중략: 473행 생략) ...
        988 | Student_988 | HR    | 4.80
        989 | Student_989 | HR    | 4.90
(495개 행)
```

문항 17 수강 기록이 있는 학생 (EXISTS)

종합실습2 문항 17 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 17. 수강(enroll) 기록이 있는 학생만 (EXISTS)
--   기대 행 수: 900 (문항 6과 동일 집합)
-- ------------------------------------------------------------
SELECT s.student_id, s.name
FROM lab.student s
WHERE EXISTS (
    SELECT 1
    FROM lab.enroll e
    WHERE e.student_id = s.student_id
)
ORDER BY s.student_id;
 student_id |    name
------------+-------------
          1 | Student_1
          2 | Student_2
          3 | Student_3
          4 | Student_4
          5 | Student_5
          6 | Student_6
          7 | Student_7
          8 | Student_8
          9 | Student_9
         11 | Student_11
         12 | Student_12
         13 | Student_13
         14 | Student_14
         15 | Student_15
         16 | Student_16
         17 | Student_17
         18 | Student_18
         19 | Student_19
         21 | Student_21
         22 | Student_22
       ... (중략: 878행 생략) ...
        998 | Student_998
        999 | Student_999
(900개 행)
```

문항 18 수강 기록이 없는 학생 (NOT EXISTS)

종합실습2 문항 18 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 18. 한 번도 수강하지 않은 학생 (NOT EXISTS)
--   기대 행 수: 100 (문항 5의 Anti-Join과 동일 결과)
-- ------------------------------------------------------------
SELECT s.student_id, s.name
FROM lab.student s
WHERE NOT EXISTS (
    SELECT 1
    FROM lab.enroll e
    WHERE e.student_id = s.student_id
)
ORDER BY s.student_id;
 student_id |     name
------------+--------------
         10 | Student_10
         20 | Student_20
         30 | Student_30
         40 | Student_40
         50 | Student_50
         60 | Student_60
         70 | Student_70
         80 | Student_80
         90 | Student_90
        100 | Student_100
        110 | Student_110
        120 | Student_120
        130 | Student_130
        140 | Student_140
        150 | Student_150
        160 | Student_160
        170 | Student_170
        180 | Student_180
        190 | Student_190
        200 | Student_200
      ... (중략: 78행 생략) ...
        990 | Student_990
       1000 | Student_1000
(100개 행)
```

문항 19 HR 학과와의 비교 (ANY / ALL)

종합실습2 문항 19 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 19. HR 학과 학생 일부와의 비교 데모 (ANY / ALL)
-- ------------------------------------------------------------
-- 19-1. > ANY : HR 학과의 최소 GPA(2.0)보다 높은 학생 → 967명
SELECT s.student_id, s.name, s.gpa
FROM lab.student s
WHERE s.gpa > ANY (SELECT gpa FROM lab.student WHERE major = 'HR')
ORDER BY s.student_id;
 student_id |     name     | gpa
------------+--------------+------
          1 | Student_1    | 2.10
          2 | Student_2    | 2.20
          3 | Student_3    | 2.30
          4 | Student_4    | 2.40
          5 | Student_5    | 2.50
          6 | Student_6    | 2.60
          7 | Student_7    | 2.70
          8 | Student_8    | 2.80
          9 | Student_9    | 2.90
         10 | Student_10   | 3.00
         11 | Student_11   | 3.10
         12 | Student_12   | 3.20
         13 | Student_13   | 3.30
         14 | Student_14   | 3.40
         15 | Student_15   | 3.50
         16 | Student_16   | 3.60
         17 | Student_17   | 3.70
         18 | Student_18   | 3.80
         19 | Student_19   | 3.90
         20 | Student_20   | 4.00
       ... (중략: 945행 생략) ...
        999 | Student_999  | 2.90
       1000 | Student_1000 | 3.00
(967개 행)

-- 19-2. > ALL : HR 학과의 최대 GPA(4.9)보다 높은 학생 → 0명 (의도된 결과)
SELECT s.student_id, s.name, s.gpa
FROM lab.student s
WHERE s.gpa > ALL (SELECT gpa FROM lab.student WHERE major = 'HR')
ORDER BY s.student_id;
 student_id | name | gpa
------------+------+-----
(0개 행)
```

문항 20 CS 학과 U DB 수강 학생 (UNION)

종합실습2 문항 20 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록 (UNION, 중복 제거)
-- ------------------------------------------------------------
SELECT s.student_id, s.name
FROM lab.student s
WHERE s.major = 'CS'
UNION
SELECT s.student_id, s.name
FROM lab.student s
JOIN lab.enroll e
  ON e.student_id = s.student_id
 AND e.course     = 'DB'
ORDER BY student_id;
 student_id |    name
------------+-------------
          5 | Student_5
         10 | Student_10
         15 | Student_15
         19 | Student_19
         20 | Student_20
         25 | Student_25
         30 | Student_30
         35 | Student_35
         39 | Student_39
         40 | Student_40
         41 | Student_41
         45 | Student_45
         50 | Student_50
         55 | Student_55
         60 | Student_60
         61 | Student_61
         62 | Student_62
         65 | Student_65
         70 | Student_70
         75 | Student_75
      ... (중략: 268행 생략) ...
        985 | Student_985
        986 | Student_986
(290개 행)
```

문항 21 ROLLUP — 학과별·GPA 구간별 인원 + 소계/총계

종합실습2 문항 21 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 21. 학과별·GPA 구간별 인원 집계 + 소계/총계 (ROLLUP)
--   - gpa_tier 파생: 3.0 미만 / 3.0~3.5 / 3.5 초과
--   - GROUP BY ROLLUP(major, gpa_tier)
--   - GROUPING(major)로 총계 행에 '전체' 라벨
--   - major, gpa_tier 순 정렬 + 소계·총계는 하단 배치
--   기대 행 수: 25 (상세 18 + 학과 소계 6 + 총계 1), 총계 1,000명
-- ------------------------------------------------------------
WITH s AS (
    SELECT major,
           CASE WHEN gpa < 3.0  THEN '3.0 미만'
                WHEN gpa <= 3.5 THEN '3.0~3.5'
                ELSE                 '3.5 초과'
           END AS gpa_tier
    FROM lab.student
)
SELECT CASE WHEN GROUPING(major)    = 1 THEN '전체'   ELSE major    END AS major,
       CASE WHEN GROUPING(gpa_tier) = 1 THEN '(소계)' ELSE gpa_tier END AS gpa_tier,
       COUNT(*) AS student_count
FROM s
GROUP BY ROLLUP(major, gpa_tier)
ORDER BY GROUPING(major),          -- 총계(전체)를 맨 하단으로
         major,
         GROUPING(gpa_tier),       -- 각 학과의 소계를 해당 학과 하단으로
         gpa_tier;
```

major	gpa_tier	student_count
BIO	3.0 미만	66
BIO	3.0~3.5	33
BIO	3.5 초과	97
BIO	(소계)	196
CE	3.0 미만	66
CE	3.0~3.5	33
CE	3.5 초과	97
CE	(소계)	196
CS	3.0 미만	65
CS	3.0~3.5	66
CS	3.5 초과	65
CS	(소계)	196
EE	3.0 미만	66
EE	3.0~3.5	33
EE	3.5 초과	97
EE	(소계)	196
HR	3.0 미만	10
HR	3.0~3.5	1
HR	3.5 초과	9
HR	(소계)	20
ME	3.0 미만	66
ME	3.0~3.5	33
ME	3.5 초과	97
ME	(소계)	196
전체	(소계)	1000

(25개 행)

문항 22 WITH RECURSIVE — 조직 계층 경로·깊이 + direct_reports

종합실습2 문항 22 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 22. WITH RECURSIVE — 조직 계층 경로와 깊이
--   - depth: 0=CEO, 1=매니저, 2=개발자
--   - path 예: 'CEO > Mgr_2 > Dev_15'
--   기대 행 수: 311
--   ※ 실측 depth 분포는 0:1 / 1:40 / 2:270 —
--     적재 수식상 Dev 30명이 CEO 직속으로 배정되어 슬라이드 배경(1/10/300)과 다름
-- ------------------------------------------------------------
WITH RECURSIVE org AS (
    SELECT emp_id, name, 0 AS depth, name::text AS path
    FROM lab.emp
    WHERE manager_id IS NULL
    UNION ALL
    SELECT e.emp_id, e.name, o.depth + 1, o.path || ' > ' || e.name
    FROM lab.emp e
    JOIN org o ON e.manager_id = o.emp_id
)
SELECT emp_id, name, depth, path
FROM org
ORDER BY path;
 emp_id |  name   | depth |          path
--------+---------+-------+------------------------
      1 | CEO     |     0 | CEO
    102 | Dev_102 |     1 | CEO > Dev_102
    112 | Dev_112 |     1 | CEO > Dev_112
     12 | Dev_12  |     1 | CEO > Dev_12
    122 | Dev_122 |     1 | CEO > Dev_122
    132 | Dev_132 |     1 | CEO > Dev_132
    142 | Dev_142 |     1 | CEO > Dev_142
    152 | Dev_152 |     1 | CEO > Dev_152
    162 | Dev_162 |     1 | CEO > Dev_162
    172 | Dev_172 |     1 | CEO > Dev_172
    182 | Dev_182 |     1 | CEO > Dev_182
    192 | Dev_192 |     1 | CEO > Dev_192
    202 | Dev_202 |     1 | CEO > Dev_202
    212 | Dev_212 |     1 | CEO > Dev_212
     22 | Dev_22  |     1 | CEO > Dev_22
    222 | Dev_222 |     1 | CEO > Dev_222
    232 | Dev_232 |     1 | CEO > Dev_232
    242 | Dev_242 |     1 | CEO > Dev_242
    252 | Dev_252 |     1 | CEO > Dev_252
    262 | Dev_262 |     1 | CEO > Dev_262
      ... (중략: 289행 생략) ...
     80 | Dev_80  |     2 | CEO > Mgr_9 > Dev_80
     90 | Dev_90  |     2 | CEO > Mgr_9 > Dev_90
(311개 행)

-- 22-보조. 매니저별 직속 부하 직원 수 (direct_reports)
--   ※ 실측: 적재 수식이 manager_id = 1+((gs-1)%10) 이어서
--     Dev 30명이 CEO 직속, Mgr_11은 부하 0명 (슬라이드 배경과 다름)
--     → 부하 0명인 Mgr_11도 보이도록 LEFT JOIN 사용
SELECT m.emp_id, m.name, COUNT(e.emp_id) AS direct_reports
FROM lab.emp m
LEFT JOIN lab.emp e ON e.manager_id = m.emp_id
WHERE m.manager_id IS NULL          -- CEO
   OR m.name LIKE 'Mgr\_%'          -- 매니저 10명
GROUP BY m.emp_id, m.name
ORDER BY m.emp_id;
 emp_id |  name   | direct_reports
--------+---------+----------------
      1 | CEO     |             40
      2 | Mgr_2   |             30
      3 | Mgr_3   |             30
      4 | Mgr_4   |             30
      5 | Mgr_5   |             30
      6 | Mgr_6   |             30
      7 | Mgr_7   |             30
      8 | Mgr_8   |             30
      9 | Mgr_9   |             30
     10 | Mgr_10  |             30
     11 | Mgr_11  |              0
(11개 행)
```

문항 23 Window Function — 학과별 GPA TOP 3 (CTE·서브쿼리)

종합실습2 문항 23 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 23. 학과별 GPA 상위 3명 (Window Function)
--   - ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id)
--   - RANK() / DENSE_RANK() 동점 처리 비교
--   - COUNT() OVER (PARTITION BY major) → total_in_major
--   기대 행 수: 18 (6개 학과 × 3명)
-- ------------------------------------------------------------
-- 23-1. CTE 방식
WITH ranked AS (
    SELECT student_id, name, major, gpa,
           ROW_NUMBER()  OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS row_no,
           RANK()        OVER (PARTITION BY major ORDER BY gpa DESC)             AS rank_no,
           DENSE_RANK()  OVER (PARTITION BY major ORDER BY gpa DESC)             AS dense_rank_no,
           COUNT(*)      OVER (PARTITION BY major)                               AS total_in_major
    FROM lab.student
)
SELECT student_id, name, major, gpa, row_no, rank_no, dense_rank_no, total_in_major
FROM ranked
WHERE row_no <= 3
ORDER BY major, row_no;
 student_id |    name     | major | gpa  | row_no | rank_no | dense_rank_no | total_in_major
------------+-------------+-------+------+--------+---------+---------------+----------------
         29 | Student_29  | BIO   | 4.90 |      1 |       1 |             1 |            196
         59 | Student_59  | BIO   | 4.90 |      2 |       1 |             1 |            196
         89 | Student_89  | BIO   | 4.90 |      3 |       1 |             1 |            196
         28 | Student_28  | CE    | 4.80 |      1 |       1 |             1 |            196
         58 | Student_58  | CE    | 4.80 |      2 |       1 |             1 |            196
         88 | Student_88  | CE    | 4.80 |      3 |       1 |             1 |            196
         25 | Student_25  | CS    | 4.50 |      1 |       1 |             1 |            196
         55 | Student_55  | CS    | 4.50 |      2 |       1 |             1 |            196
         85 | Student_85  | CS    | 4.50 |      3 |       1 |             1 |            196
         26 | Student_26  | EE    | 4.60 |      1 |       1 |             1 |            196
         56 | Student_56  | EE    | 4.60 |      2 |       1 |             1 |            196
         86 | Student_86  | EE    | 4.60 |      3 |       1 |             1 |            196
        989 | Student_989 | HR    | 4.90 |      1 |       1 |             1 |             20
        988 | Student_988 | HR    | 4.80 |      2 |       2 |             2 |             20
        987 | Student_987 | HR    | 4.70 |      3 |       3 |             3 |             20
         27 | Student_27  | ME    | 4.70 |      1 |       1 |             1 |            196
         57 | Student_57  | ME    | 4.70 |      2 |       1 |             1 |            196
         87 | Student_87  | ME    | 4.70 |      3 |       1 |             1 |            196
(18개 행)

-- 23-2. 서브쿼리(인라인 뷰) 방식
SELECT student_id, name, major, gpa, row_no, rank_no, dense_rank_no, total_in_major
FROM (
    SELECT student_id, name, major, gpa,
           ROW_NUMBER()  OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS row_no,
           RANK()        OVER (PARTITION BY major ORDER BY gpa DESC)             AS rank_no,
           DENSE_RANK()  OVER (PARTITION BY major ORDER BY gpa DESC)             AS dense_rank_no,
           COUNT(*)      OVER (PARTITION BY major)                               AS total_in_major
    FROM lab.student
) ranked
WHERE row_no <= 3
ORDER BY major, row_no;
 student_id |    name     | major | gpa  | row_no | rank_no | dense_rank_no | total_in_major
------------+-------------+-------+------+--------+---------+---------------+----------------
         29 | Student_29  | BIO   | 4.90 |      1 |       1 |             1 |            196
         59 | Student_59  | BIO   | 4.90 |      2 |       1 |             1 |            196
         89 | Student_89  | BIO   | 4.90 |      3 |       1 |             1 |            196
         28 | Student_28  | CE    | 4.80 |      1 |       1 |             1 |            196
         58 | Student_58  | CE    | 4.80 |      2 |       1 |             1 |            196
         88 | Student_88  | CE    | 4.80 |      3 |       1 |             1 |            196
         25 | Student_25  | CS    | 4.50 |      1 |       1 |             1 |            196
         55 | Student_55  | CS    | 4.50 |      2 |       1 |             1 |            196
         85 | Student_85  | CS    | 4.50 |      3 |       1 |             1 |            196
         26 | Student_26  | EE    | 4.60 |      1 |       1 |             1 |            196
         56 | Student_56  | EE    | 4.60 |      2 |       1 |             1 |            196
         86 | Student_86  | EE    | 4.60 |      3 |       1 |             1 |            196
        989 | Student_989 | HR    | 4.90 |      1 |       1 |             1 |             20
        988 | Student_988 | HR    | 4.80 |      2 |       2 |             2 |             20
        987 | Student_987 | HR    | 4.70 |      3 |       3 |             3 |             20
         27 | Student_27  | ME    | 4.70 |      1 |       1 |             1 |            196
         57 | Student_57  | ME    | 4.70 |      2 |       1 |             1 |            196
         87 | Student_87  | ME    | 4.70 |      3 |       1 |             1 |            196
(18개 행)
```

문항 24 LAG — 이전 과목 대비 성적 변화

종합실습2 문항 24 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 24. LAG() — 학생별 이전 수강 과목 대비 성적 변화
--   - grade → 점수 변환: A=4, B=3, C=2, D=1
--   - LAG(score) OVER (PARTITION BY student_id ORDER BY course)
--   - diff = 현재 점수 - 이전 점수, 상승/유지/하락 표시
--   - score_range = 학생별 최고점 - 최저점 (Window Function)
--   기대 행 수: 2,302 (enroll 전체)
-- ------------------------------------------------------------
SELECT student_id, course, score,
       LAG(score) OVER w                AS prev_score,
       score - LAG(score) OVER w        AS diff,
       CASE
           WHEN LAG(score) OVER w IS NULL       THEN '(첫 과목)'
           WHEN score - LAG(score) OVER w > 0   THEN '상승'
           WHEN score - LAG(score) OVER w = 0   THEN '유지'
           ELSE                                      '하락'
       END AS trend,
       MAX(score) OVER (PARTITION BY student_id)
     - MIN(score) OVER (PARTITION BY student_id) AS score_range
FROM (
    SELECT student_id, course,
           CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3
                      WHEN 'C' THEN 2 WHEN 'D' THEN 1 END AS score
    FROM lab.enroll
) e
WINDOW w AS (PARTITION BY student_id ORDER BY course)
ORDER BY student_id, course;
 student_id |  course   | score | prev_score | diff |   trend   | score_range
------------+-----------+-------+------------+------+-----------+-------------
          1 | Course_3  |     2 |            |      | (첫 과목) |           3
          1 | Course_4  |     1 |          2 |   -1 | 하락      |           3
          1 | Course_5  |     4 |          1 |    3 | 상승      |           3
          2 | Course_4  |     1 |            |      | (첫 과목) |           3
          2 | Course_5  |     4 |          1 |    3 | 상승      |           3
          3 | Course_5  |     4 |            |      | (첫 과목) |           2
          3 | Course_6  |     3 |          4 |   -1 | 하락      |           2
          3 | Course_7  |     2 |          3 |   -1 | 하락      |           2
          4 | Course_6  |     3 |            |      | (첫 과목) |           1
          4 | Course_7  |     2 |          3 |   -1 | 하락      |           1
          5 | Course_7  |     2 |            |      | (첫 과목) |           3
          5 | Course_8  |     1 |          2 |   -1 | 하락      |           3
          5 | Course_9  |     4 |          1 |    3 | 상승      |           3
          6 | Course_8  |     1 |            |      | (첫 과목) |           3
          6 | Course_9  |     4 |          1 |    3 | 상승      |           3
          7 | Course_10 |     3 |            |      | (첫 과목) |           2
          7 | Course_11 |     2 |          3 |   -1 | 하락      |           2
          7 | Course_9  |     4 |          2 |    2 | 상승      |           2
          8 | Course_10 |     3 |            |      | (첫 과목) |           1
          8 | Course_11 |     2 |          3 |   -1 | 하락      |           1
      ... (중략: 2280행 생략) ...
       1001 | AI        |     4 |            |      | (첫 과목) |           0
       1010 | ML        |     3 |            |      | (첫 과목) |           0
(2302개 행)
```

문항 25 누적합·3개 이동평균 (ROWS BETWEEN) + 50% 지점

종합실습2 문항 25 — psql 실행 (skala_db, lab 스키마)

```
-- ------------------------------------------------------------
-- 문항 25. orders 누적 주문금액과 3개 주문 이동평균 (ROWS BETWEEN)
--   - 전체 누적합: SUM() OVER (ORDER BY order_id ROWS BETWEEN
--                  UNBOUNDED PRECEDING AND CURRENT ROW)
--   - 3개 이동평균: AVG() OVER (... ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
--   - 고객별 누적 구매금액: PARTITION BY customer_id
--   기대 행 수: 3,000
-- ------------------------------------------------------------
SELECT order_id, customer_id, amount,
       SUM(amount) OVER (ORDER BY order_id
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
       ROUND(AVG(amount) OVER (ORDER BY order_id
                               ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS moving_avg_3,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_id
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cust_running_total
FROM lab.orders
ORDER BY order_id;
 order_id | customer_id | amount  | running_total | moving_avg_3 | cust_running_total
----------+-------------+---------+---------------+--------------+--------------------
        1 |           2 |   18.01 |         18.01 |        18.01 |              18.01
        2 |           3 |   31.02 |         49.03 |        24.52 |              31.02
        3 |           4 |   44.03 |         93.06 |        31.02 |              44.03
        4 |           5 |   57.04 |        150.10 |        44.03 |              57.04
        5 |           6 |   70.05 |        220.15 |        57.04 |              70.05
        6 |           7 |   83.06 |        303.21 |        70.05 |              83.06
        7 |           8 |   96.07 |        399.28 |        83.06 |              96.07
        8 |           9 |  109.08 |        508.36 |        96.07 |             109.08
        9 |          10 |  122.09 |        630.45 |       109.08 |             122.09
       10 |          11 |  135.10 |        765.55 |       122.09 |             135.10
       11 |          12 |  148.11 |        913.66 |       135.10 |             148.11
       12 |          13 |  161.12 |       1074.78 |       148.11 |             161.12
       13 |          14 |  174.13 |       1248.91 |       161.12 |             174.13
       14 |          15 |  187.14 |       1436.05 |       174.13 |             187.14
       15 |          16 |  200.15 |       1636.20 |       187.14 |             200.15
       16 |          17 |  213.16 |       1849.36 |       200.15 |             213.16
       17 |          18 |  226.17 |       2075.53 |       213.16 |             226.17
       18 |          19 |  239.18 |       2314.71 |       226.17 |             239.18
       19 |          20 |  252.19 |       2566.90 |       239.18 |             252.19
       20 |          21 |  265.20 |       2832.10 |       252.19 |             265.20
     ... (중략: 2978행 생략) ...
     2999 |         500 | 3992.99 |    7361980.00 |      3979.98 |           16457.94
     3000 |           1 | 4005.00 |    7365985.00 |      3992.66 |           16530.00
(3000개 행)

-- 25-보조. 누적합이 전체 합의 50%를 초과하는 첫 번째 order_id
WITH r AS (
    SELECT order_id,
           SUM(amount) OVER (ORDER BY order_id
                             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_total,
           SUM(amount) OVER () AS grand_total
    FROM lab.orders
)
SELECT order_id, running_total, grand_total
FROM r
WHERE running_total > grand_total * 0.5
ORDER BY order_id
LIMIT 1;
 order_id | running_total | grand_total
----------+---------------+-------------
     1503 |    3686585.56 |  7365985.00
(1개 행)
```